# Svelto-DNA: Interactive Splice-Disruption Analysis on a Frozen Genomic Foundation Model with Speculative In-Silico Mutagenesis

Derrick Rajkumar Yuvaraj R.

Abstract—Splice-altering variants are a frequent and underdiagnosed cause of genetic disease. Deep sequence models can score them, but exhaustive in-silico mutagenesis (ISM) of a region needs one forward pass per candidate substitution, which makes interactive exploration slow. We present Svelto-DNA, an open system that pairs a frozen HyenaDNA backbone with a small trained splice-site head, a validated vectorized splice-disruption ($\Delta$) calculator, a WebSocket scoring service and a browser-based explorer. Because HyenaDNA is causal, the head reads forward and reverse-complement features, so every position sees both flanks. On held-out chromosomes (62.7 M positions, 112,019 annotated sites) the model reaches donor/acceptor PR-AUC 0.630/0.552 and ROC-AUC 0.9989/0.9984 while training only 0.2 M parameters for 14 minutes on a laptop GPU. Fed SpliceAI's own probability tracks, our $\Delta$ calculator reproduces published SpliceAI scores for eight ClinVar pathogenic variants within their 3-decimal rounding, and our own model calls the correct site loss for all eight (mean peak $\Delta$ 0.944 versus SpliceAI's 0.973). To accelerate ISM we study a speculative draft-and-verify scheme: one reference pass drafts which candidates could matter and only the top-$K$ are verified exactly in batched passes. We derive a cost model that predicts the measured speedups within 1.4% and show that, on a laptop GPU, exact batching gives only 1.2×; every larger speedup comes from verifying fewer candidates and costs recall (2.4× at 0.83 recall of high-impact variants, 9.1× at 0.31). The heuristic draft is 1.7–4.5× better than random selection but reaches only 51–87% of an oracle. We report these trade-offs, and two negative results, so that speculative claims for genomic models can be judged against recall rather than speed alone.

Index Terms—RNA splicing, variant effect prediction, genomic foundation models, HyenaDNA, in-silico mutagenesis, speculative decoding, algorithm analysis, ClinVar

## I. Introduction

Pre-mRNA splicing removes introns and joins exons, and it is directed largely by short sequence signals. Almost every intron begins with GT at its 5′ end (the donor site) and ends with AG at its 3′ end (the acceptor site), and both dinucleotides sit inside longer, degenerate consensus motifs that are complemented by a branch point, a polypyrimidine tract and exonic and intronic regulatory elements [1], [2]. A single-nucleotide variant (SNV) that destroys one of these signals, or creates a cryptic one, can cause exon skipping, intron retention or a shifted exon boundary, and therefore a truncated or altered protein. Such variants are a recurring cause of inherited disease [3], [4]. A classic example is the MAPT gene, where mutations at the exon 10 donor change the ratio of tau isoforms and cause frontotemporal dementia with parkinsonism (FTDP-17) [5].

Interpreting splice variants is hard in practice. Variants at the canonical GT/AG dinucleotides are usually clear, but most variants lie elsewhere, and their effect depends on context that a human reviewer cannot judge by eye. Clinical guidelines therefore admit computational evidence [6], and recent recommendations explicitly calibrate splice predictors for this purpose [7]. Deep models now predict splicing from sequence with high accuracy: SpliceAI [8] and Pangolin [9] score a variant by the change ($\Delta$) in predicted site probabilities within a local window. In parallel, genomic foundation models such as DNABERT [10], [11], the Nucleotide Transformer [12] and HyenaDNA [13] learn general-purpose sequence representations that can be adapted to downstream tasks with little labelled data.

Using these models interactively remains slow. A clinician or researcher who wants to know which positions in a region are fragile must perform in-silico saturation mutagenesis (ISM) [14]: every alternative base at every position, each scored with its own forward pass. For $n$ positions that is $3n$ passes over a sequence that itself grows with $n$, so the cost is super-linear in the region size. Incremental methods such as fastISM [15] exploit the local receptive fields of convolutional networks, but long-convolution models such as HyenaDNA mix information across the whole sequence through FFT convolutions, so a single-base change alters every downstream state and the incremental trick does not apply.

Large language models faced a similar problem with autoregressive decoding, and speculative decoding [16]–[18] and multi-head drafting [19] reduce latency by drafting cheaply and verifying in parallel. We ask whether the same idea helps variant scanning. A cheap draft ranks candidate variants by how likely they are to matter, and the expensive model verifies only the top-$K$. The answer turns out to be more nuanced than in language modelling. Token-level speculative decoding is lossless because rejected drafts are regenerated, but in variant scanning an unverified candidate is simply unknown, so every speedup must be weighed against the high-impact variants it misses.

To answer the question honestly we built a complete

system and measured it end to end. Our contributions are:

- an open data pipeline from GENCODE, GRCh38 and ClinVar to chromosome-disjoint splice-site datasets, including the discovery and repair of two silent labelling errors (Section V);
- a splice-site head on a frozen HyenaDNA backbone that concatenates forward and reverse-complement features to compensate for the backbone's causality, which we verify empirically (Section VI);
- a vectorized $\Delta$ calculator validated against published SpliceAI scores for eight ClinVar pathogenic variants (Section VII);
- speculative ISM with exact batched verification, a complexity analysis, a cost model that predicts measured speedups within 1.4%, and recall bounds for random and oracle drafts (Sections VIII and IX);
- an interactive WebSocket service and browser explorer with sub-15 ms edit latency (Section X);
- an evaluation that reports recall next to every speedup, two negative results, and the measurement conditions under which every timing was taken (Sections XII and XIII).

The rest of the paper is organised as follows. Section II gives background and Section III discusses related work. Section IV formalises the problem. Sections V–X describe the data, model, $\Delta$ calculator, speculative ISM, its complexity and the interactive system. Section XI gives the experimental setup, Section XII the results, and Sections XIII and XIV the discussion and limitations.

## II. Background

### A. Splice sites and splice-disrupting variants

In the human genome the donor consensus spans roughly the last three exonic and first six intronic bases (MAG|GTRAGT, where | marks the exon–intron boundary), and the acceptor consensus consists of a polypyrimidine tract followed by YAG|G [1]. Only the GT and AG dinucleotides are nearly invariant, and the remaining positions contribute graded information [20]. Variants act in four ways that map directly onto the $\Delta$ components used later: they can weaken an existing donor (donor loss) or acceptor (acceptor loss), or strengthen a cryptic one (donor gain, acceptor gain).

### B. The SpliceAI $\Delta$ score

SpliceAI predicts, for every position, the probability that it is a donor, an acceptor or neither. A variant is scored by running the model on the reference and the mutant sequence and taking, within a window of $\pm W$ bases around the variant, the largest increase (gain) and decrease (loss) of donor and acceptor probability; the published scores use $W = 50$ [8]. The four scores lie in $[0, 1]$, and cut-offs of 0.2, 0.5 and 0.8 are commonly used as high-recall, recommended and high-precision thresholds [7], [8]. We use exactly this definition (Section VII) so that our scores are comparable with published ones.

### C. Long-convolution genomic foundation models

Transformers [21] scale quadratically with sequence length, which limits the context of attention-based DNA models such as DNABERT [10] and leads them to use $k$-mer or byte-pair tokens [11], [12]. The Hyena operator [22] replaces attention with data-controlled gating and implicitly parameterised long convolutions evaluated with the FFT [23], so a layer costs $O(L \log L)$ in the sequence length $L$. HyenaDNA [13] applies this to the human genome with single-nucleotide tokens and contexts of up to one million bases. Its convolutions are causal: the state at position $t$ depends only on bases at positions $\leq t$. This suits next-token pre-training but is a mismatch for splice prediction, where a donor is defined as much by the intron downstream as by the exon upstream. State-space models such as Mamba [24] share the same causality, and Caduceus [25] addresses it with bidirectional, reverse-complement-equivariant layers. We take the simpler route of running a frozen causal model on both strands (Section VI).

### D. Speculative inference

Speculative decoding [16], [17] uses a small draft model to propose several tokens and the large target model to verify them in one parallel pass; accepted tokens are kept and the first rejected one is resampled, so the output distribution is unchanged. Blockwise parallel decoding [18] and Medusa [19] draft with extra prediction heads instead of a separate model, and Medusa verifies a tree of candidates with a tree-shaped attention mask. Speedups come from the fact that verification is parallel while generation is sequential, and the key quantity is the acceptance rate $\alpha$ of drafted tokens. Section VIII adapts these ideas to variant scanning, where the structure of the problem is different.

## III. Related Work

Splice prediction. Early tools scored short motifs: MaxEntScan [20] models 9-mer donors and 23-mer acceptors with maximum-entropy distributions. The splicing code of Xiong et al. [4] predicted exon inclusion from sequence features with deep networks, and MMSplice [26] combined modules for exon, intron and splice-site regions. SpliceAI [8] trained a deep dilated residual CNN on 10 kb of context end to end and introduced the $\Delta$ score, and Pangolin [9] extended this to tissue-specific splice-site usage across species. Broad's SpliceAI-lookup [27] serves SpliceAI and Pangolin scores on demand, which we use as ground truth. General variant-effect scores such as CADD [28] integrate many annotations but are not specific to splicing. Our model is weaker than SpliceAI, which we treat as a reference rather than a competitor; our aim is a system that is fast enough to explore interactively and honest about its trade-offs.

Genomic foundation models. DNABERT [10] pre-trained a BERT encoder on overlapping 6-mers, DNABERT-2 [11] switched to byte-pair encoding and multi-species data, and the Nucleotide Transformer [12] scaled attention-based models to billions of parameters across many genomes. HyenaDNA [13] trades attention for long convolutions and single-nucleotide resolution, and Caduceus [25] adds bidirectionality and reverse-complement equivariance on top of Mamba [24]. Enformer [29] combines convolutions and attention for expression prediction. These models are usually evaluated by fine-tuning on classification benchmarks; we instead use a frozen backbone inside an interactive tool and measure the latency that results.

Accelerating in-silico mutagenesis. ISM is the standard way to read variant effects out of a sequence model [14]. fastISM [15] speeds it up for CNNs by recomputing only the positions inside a variant's receptive field. Gradient-based attributions [30]–[32] approximate ISM with one backward pass but are first-order and can miss strongly non-linear effects, such as destroying a GT. FFT-based long convolutions can be accelerated at the kernel level [33], which is complementary to our work. To our knowledge, speculative draft-and-verify has not been studied for ISM.

Speculative inference. Beyond the work above [16]–[19], the common premise is that verifying many items in parallel costs little more than processing one. We show that for small genomic models on consumer GPUs this premise is weak, because a batched pass is barely cheaper per sequence than a single one.

Interactive genomics tools. Genome browsers such as UCSC [34] display precomputed tracks, and SpliceAI-lookup [27] scores single variants on request. We are not aware of a tool that rescores a region's splice landscape live while the user edits bases, which is what our WebSocket service provides.

## IV. Problem Formulation

Let $x \in \{\mathrm{A},\mathrm{C},\mathrm{G},\mathrm{T}\}^L$ be a DNA window in transcript orientation and let $f$ be a model that maps it to per-position probabilities $p = f(x) \in [0,1]^{L\times 3}$ over the classes {neither, donor, acceptor}. A candidate $c = (v, b)$ substitutes base $b \neq x_v$ at position $v$; we write $x^c$ for the mutated sequence.

Definition 1 (Splice disruption). The disruption of $c$ is $\Delta(c) = \max_{k\in\{\mathrm{d},\mathrm{a}\}} \max_{|j-v|\le W} |f(x^c)_{j,k} - f(x)_{j,k}|$, the largest change in donor or acceptor probability within $\pm W$ bases of the variant. It equals the maximum of the four SpliceAI components in Section VII.

Definition 2 (Saturation scan). For a region $R = [s, e)$ of $n = e - s$ positions the candidate set is $C = \{(v, b) : v \in R,\ b \neq x_v\}$, with $N = |C| = 3n$. Given a threshold $\tau$, the high-impact set is $H = \{c \in C : \Delta(c) \ge \tau\}$. Exhaustive ISM computes $\Delta(c)$ for all $c \in C$.

A speculative scan with budget $K$ computes $\Delta$ exactly for a subset $S \subseteq C$ with $|S| = K$, chosen without evaluating $f$ on any mutant, and reports the rest as unknown. Its quality is measured by

$$\text{recall} = \frac{|S \cap H|}{|H|}, \qquad \alpha = \frac{|S \cap H|}{|S|}, \tag{1}$$

and its cost by wall-clock time and the fraction of forward passes saved, $1 - (1 + K)/(1 + N)$, where the extra pass is the reference. The goal is to maximise recall for a given time budget, or to minimise time for a required recall. Throughout we use $W = 50$ and $\tau = 0.2$, SpliceAI's high-recall cut-off.

## V. Data

### A. Annotation and sequence

We use the GENCODE v46 basic annotation [35] and keep one canonical protein-coding transcript per gene, identified by the Ensembl_canonical tag. This gives 19,120 transcripts on chr1–22 with 366,000 splice sites. Donor and acceptor coordinates are derived from exon boundaries, and sequence comes from the Ensembl GRCh38 primary assembly [36]. Each chromosome is decompressed once into an uppercase single-line cache file and read through memory mapping, so fetching a window is a direct slice that needs no parsing.

### B. Training windows

Algorithm 1 shows how windows are built. Each transcript's span is tiled into consecutive 1 kb blocks in transcript ($5'\to3'$) orientation, and each block is extended by 1 kb of flank on both sides, giving 3 kb of input of which only the central 1 kb is labelled. Minus-strand transcripts are reverse-complemented, so the model always sees the sense strand. A site is labelled on the first base of its dinucleotide: the G of GT for donors and the A of AG for acceptors. All blocks that contain a site are kept, plus one randomly chosen site-free block per transcript (seed 42), so positives are enriched relative to the genome while the model still sees intronic background.

### C. Chromosome splits

Splits are by chromosome, with no chromosome or gene shared between them (Table I). We train on chr11–22, select checkpoints on chr2, 4, 6, 8 and 10, and test on chr1, 3, 5, 7 and 9, the test chromosomes used by SpliceAI. Paralogs are not removed, which we list as a limitation. The test set contains 62,739 windows, i.e. 62.7 M labelled positions with 56,008 donors and 56,011 acceptors, a non-site to site ratio of 559:1.

### D. ClinVar variants and two silent errors

From the ClinVar GRCh38 release [37] we keep SNVs within ±50 bp of an annotated splice site. Distances are computed with a per-chromosome array of all site positions, sorted once, and a binary search per variant: $O(S \log S)$ to build for $S$ sites and $O(\log S)$ per query,

**Algorithm 1** Transcript-oriented window construction

**Require:** transcripts $\mathcal{T}$, genome $G$, block $B$=1000, flank $F$=1000

```
for t ∈ T do
    Q ← oriented first-base positions of all sites of t
    B ← {t.start, t.start+B, ...}                ▷ block starts
    keep each b ∈ B with Q ∩ [b, b+B) ≠ ∅
    keep one random site-free block
    for each kept block b do
        y ← labels on [b, b+B) from Q            ▷ 0/1/2
        x ← G.fetch(t.chrom, b−F, b+B+F)
        if t.strand = − then x ← rc(x); y ← reverse(y)
        end if
        emit (x, y)
    end for
end for
```

TABLE I
Dataset statistics (GENCODE v46 canonical protein-coding transcripts, GRCh38)

| Split | Chromosomes | Windows | Labelled positions |
|---|---|---|---|
| Train | 11–22 | 86,811 | 86.8 M |
| Validation | 2, 4, 6, 8, 10 | 51,657 | 51.7 M |
| Test | 1, 3, 5, 7, 9 | 62,739 | 62.7 M |

19,120 transcripts, 366,000 sites
window: 1 kb labelled block + 1 kb flank on each side

so annotating 2.1 M variants takes seconds. Variants are labelled by clinical significance (Table II).

Two silent errors were found while building this pipeline, and both would have corrupted downstream evaluation without raising an error. First, the original parser read the distance to the nearest exon junction from a field that real ClinVar records do not have, and defaulted it to zero. Every SNV in ClinVar therefore passed the ±50 bp filter. Distances are now computed from the annotation as described above. Second, recent ClinVar releases describe disputed variants as "Conflicting classifications of pathogenicity". That text contains the word "pathogenic", and a substring match labelled 86,949 conflicting variants as pathogenic. Conflicting submissions are now never used as labels, whatever words they contain, and a regression test covers the exact ClinVar wording. Both errors are of a kind that aggregate metrics hide, which is why we report them.

## VI. Model

### A. Frozen backbone

The backbone is HyenaDNA-small-32k (LongSafari/hyenadna-small-32k-seqlen-hf): 4 Hyena layers with hidden size $D = 256$ and 3,277,568 parameters, pre-trained on the human reference genome with a 32 k context. We load it with its own single-nucleotide vocabulary and without special tokens, which would otherwise shift every output by one position, and we keep all of its parameters frozen. The checkpoint's remote code was audited before use. Weights stay in fp32 and inference runs under fp16 autocast [38], which matches fp32 hidden states with cosine similarity 0.99998 on a 10 kb window. Keeping the stored weights in fp32 preserves the accuracy of the FFT convolution filters.

TABLE II
ClinVar GRCh38 SNVs within ±50 bp of a canonical splice site

| Label | Variants |
|---|---|
| Pathogenic / likely pathogenic | 119,787 |
| Benign / likely benign | 732,647 |
| Held out (uncertain or conflicting) | 1,266,142 |
| Total | 2,118,576 |
| Conflicting, previously mislabelled pathogenic | 86,949 |

### B. Reverse-complement fusion

HyenaDNA is causal, so its state $H(x)_t$ at position $t$ summarises only $x_{1..t}$. A donor at $t$, however, is defined by the intron downstream of it. We therefore run the backbone a second time on the reverse complement $\mathrm{rc}(x)$, in which the downstream sequence comes first, and flip the result back into forward coordinates:

$$h_t = \left[\, H(x)_t \;;\; H(\mathrm{rc}(x))_{L-1-t} \,\right] \in \mathbb{R}^{2D}. \tag{2}$$

The first half sees the upstream flank and the second half sees the downstream flank on the opposite strand, so together they cover both. For batches of unequal length the flip is done within each sequence's own length. We verified this directly. After a single-base edit 50 bp downstream of position $t$, the forward half of $h_t$ changes only by fp16 FFT round-off ($\sim 3\times10^{-3}$), while the reverse-complement half changes 80–300× more; a unit test asserts a ratio of at least 20.

### C. Splice head

The head maps $h_t$ to class logits with a two-hidden-layer perceptron: LayerNorm($2D$), Linear($2D \to 256$), GELU, Linear($256 \to 256$), GELU, Linear($256 \to 3$). It has 198,915 parameters, 6.1% of the backbone, and is applied independently at every position, so it adds no sequence mixing beyond what the backbone provides. Probabilities are $p_t = \mathrm{softmax}(f(h_t))$.

### D. Training

Only the head is trained. Backbone features are computed without gradients, so training stores no backbone activations. The loss is class-weighted cross-entropy over the labelled 1 kb block of each window,

$$\mathcal{L} = -\frac{1}{\sum_t w_{y_t}} \sum_t w_{y_t} \log p_{t,y_t}, \qquad w_0 = 1,\ w_1 = w_2 = 20, \tag{3}$$

which counteracts the 559:1 imbalance without resampling. We use AdamW [39] with learning rate $10^{-3}$ and

TABLE III
Model and training settings

| | |
|---|---|
| Backbone | HyenaDNA-small-32k, frozen, 3.28 M params |
| Precision | fp32 weights, fp16 autocast |
| Features | $[h^{\rightarrow}; h^{\leftarrow}]$, $2D = 512$ |
| Head | LN, 512→256→256→3, GELU; 0.20 M |
| Loss | weighted cross-entropy, site weight 20 |
| Optimiser | AdamW, lr $10^{-3}$, weight decay $10^{-2}$ |
| Schedule | one-cycle, 4,000 steps, batch 16 |
| Selection | best mean val. PR-AUC (1,500 windows) |
| Seed / time | 42 / 14.0 min |

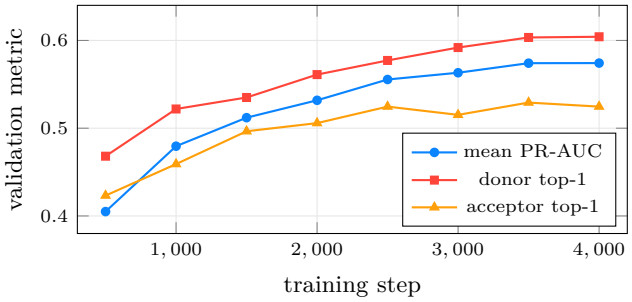



Fig. 1. Validation metrics during head training (1,500 fixed validation windows, every 500 steps). Mean ROC-AUC rises from 0.9969 to 0.9985 over the same period; PR-AUC has nearly plateaued by step 3,500.

weight decay $10^{-2}$, a one-cycle schedule [40], and 4,000 steps of 16 randomly sampled training windows, i.e. about 64,000 window presentations (0.74 epochs). Every 500 steps we evaluate on a fixed sample of 1,500 validation windows and keep the checkpoint with the best mean donor/acceptor PR-AUC. Training takes 14 minutes on the laptop GPU described in Section XI. Table III lists all settings and Fig. 1 shows the validation curve.

## VII. Variant Effect and Its Validation

### A. Definition

For a variant at position $v$ with reference and mutant probability tracks $p^{\text{ref}}, p^{\text{mut}}$ and window $W = 50$ we follow SpliceAI:

$$\Delta_{\text{DG}} = \max_{|j-v|\le W} \left(p^{\text{mut}}_{j,\text{d}} - p^{\text{ref}}_{j,\text{d}}\right), \tag{4}$$

$$\Delta_{\text{DL}} = \max_{|j-v|\le W} \left(p^{\text{ref}}_{j,\text{d}} - p^{\text{mut}}_{j,\text{d}}\right), \tag{5}$$

with $\Delta_{\text{AG}}, \Delta_{\text{AL}}$ defined analogously on the acceptor channel, and $\Delta = \max(\Delta_{\text{DG}}, \Delta_{\text{DL}}, \Delta_{\text{AG}}, \Delta_{\text{AL}})$. Each component also has a peak position, the offset $j - v$ at which its maximum is attained.

### B. Vectorized computation

Stacking the four signed differences $\pm(p^{\text{mut}} - p^{\text{ref}})$ on the donor and acceptor channels gives a $4 \times L$ tensor. A single 1D max-pool with kernel $2W + 1$, stride 1 and padding $W$ then yields all four components at every position at once, so the same code scores one variant, a batch of variants, or the full landscape displayed in the browser.

TABLE IV
Δ calculator on SpliceAI tracks vs. published SpliceAI scores. "Max dev." is over all four components; offsets are genomic.

| Gene | Str. | Peak | Ours | Published | Max dev. | Offset |
|---|---|---|---|---|---|---|
| ABAT | + | DL | 0.9960 | 0.996 | 0.000034 | −1 ✓ |
| ABCC2 | + | DL | 0.9947 | 0.995 | 0.000394 | −2 ✓ |
| ABCG8 | + | AL | 0.9365 | 0.936 | 0.000470 | +1 ✓ |
| ABHD5 | + | AL | 0.9808 | 0.981 | 0.000250 | +2 ✓ |
| AAAS | − | DL | 0.9853 | 0.985 | 0.000421 | +1 ✓ |
| AAGAB | − | DL | 0.9857 | 0.986 | 0.000343 | +1 ✓ |
| AARS2 | − | DL | 0.9052 | 0.905 | 0.000476 | +1 ✓ |
| ABCA1 | − | DL | 0.9967 | 0.997 | 0.000285 | +1 ✓ |

A naive loop costs $O(LW)$ comparisons per channel, and a monotone-deque sliding-window maximum would cost $O(L)$. The pooling kernel performs $O(LW)$ work but in parallel, and for our sizes it is bound by kernel-launch overhead rather than arithmetic: a 10 kb window takes a median of 0.21 ms on the GPU, the same as a 1 kb window, and 0.55 ms on the CPU.

### C. Validation against published SpliceAI scores

An implementation of Δ can be wrong in subtle ways, for instance with off-by-one windows, swapped gain and loss, or strand handling. To test ours independently of our model, we ran the official SpliceAI five-model ensemble on reference and mutant 11 kb windows for eight ClinVar pathogenic SNVs at canonical splice dinucleotides, four per strand, covering donors and acceptors. We fed its full ±50 bp probability tracks to our calculator and compared the result with the scores and positions published by SpliceAI-lookup [27]. Table IV shows the result. All 32 component scores agree within 0.000476, inside the ±0.0005 implied by the 3-decimal rounding of the published values, and all eight peak positions match exactly. Because the published scores are rounded, closer agreement cannot be demonstrated.

## VIII. Speculative In-Silico Mutagenesis

### A. Why tree verification does not apply

Medusa-style verification [19] checks a tree of drafted continuations in one pass by giving each branch its own attention mask. HyenaDNA has no attention, so there is no mask through which branches could share a pass; every distinct sequence needs its own row in the batch. Variant scanning also has a different structure from decoding: candidates are not sequential, each is an independent sequence, and there is nothing to "accept" in the language-model sense. We therefore move speculation from the token level to the level of whole variants: draft which variants matter, then verify those exactly.

### B. Algorithm

Algorithm 2 shows the procedure. One reference pass gives $p^{\text{ref}}$. A draft score is then computed for every

Algorithm 2 Speculative ISM

Require: sequence $x$, region $R$, model $f$, budget $K$, batch $B$

$C \leftarrow \{(v, b) : v \in R,\ b \neq x_v\}$ ▷ $N = 3|R|$

$p^{\text{ref}} \leftarrow f(x)$ ▷ one forward pass

for $c \in C$ do $d(c) \leftarrow$ draft score from $p^{\text{ref}}$ and $x$

end for

$S \leftarrow$ top-$K$ of $C$ by $d$ ▷ sort or select

$\Delta(c) \leftarrow$ NaN for all $c \in C$

for each chunk $S_i$ of $B$ candidates in $S$ do

  $P \leftarrow f(\{x^c : c \in S_i\})$ ▷ one batched pass

  for $c \in S_i$ do

    $\Delta(c) \leftarrow \max_{|j-v| \le W,\, k} |P^c_{j,k} - p^{\text{ref}}_{j,k}|$

  end for

end for

return $\Delta$

candidate from the reference prediction and the sequence alone, without running the model:

$$d(c) = \max\big(s_{\text{on}}(c),\ s_{\text{near}}(c),\ 0.5 \cdot \mathbb{1}[c \text{ creates GT or AG}]\big), \tag{6}$$

where, with $s_j = \max(p^{\text{ref}}_{j,\text{d}}, p^{\text{ref}}_{j,\text{a}})$ the reference site probability,

$$s_{\text{on}}(c) = \max(s_v, s_{v-1}), \quad s_{\text{near}}(c) = \max_{|j-v| \le 10} s_j\, e^{-|j-v|/4}. \tag{7}$$

The first term catches loss at a predicted site whose dinucleotide the variant hits (the label sits on the first base, so the second base is at $v-1$). The second catches loss through damage to the surrounding consensus, which decays over a few bases. The third catches gain of a new GT or AG. The $K$ highest-scoring candidates are mutated and verified exactly in batched forward passes of $B$ sequences, and all others are reported as unknown (NaN), never as zero.

### C. Correctness properties

The scheme has two properties that the tests check. First, verified scores are exact: a candidate's $\Delta$ does not depend on which other candidates share its batch. Batched and sequential verification agree within $6 \times 10^{-8}$ in fp32 and within $4.1 \times 10^{-3}$ under fp16 autocast, where the difference comes from the batched half-precision kernels. Second, unverified candidates are never reported as benign, so a downstream consumer can always tell "scored low" from "not scored". Neither property makes the scan exhaustive, so recall has to be measured against exhaustive ISM, which we do in Section XII.

### D. Gradient draft (negative result)

We also tried a first-order draft that scores each substitution by $|\nabla_e S \cdot (e_{\text{alt}} - e_{\text{ref}})|$, where $e$ are the token embeddings and $S$ is the total donor and acceptor probability mass near the region. It needs one backward pass instead of none. At $K = 96$ it reached recall 0.65 against the heuristic's 0.83 on 16 windows, so we did not pursue it. A plausible reason is that destroying a GT is a highly non-linear event that a linearisation around the reference does not capture.

## IX. Complexity Analysis

### A. Cost of one forward pass

A Hyena layer applies dense projections at cost $O(LD^2)$ and FFT long convolutions at cost $O(DL \log L)$ [22], [23]. With $\ell$ layers, one pass costs

$$T_f(L) = O\big(\ell\,(LD^2 + DL \log L)\big), \tag{8}$$

and our feature extractor runs two of them. In practice small inputs are dominated by fixed overheads: the measured latency of one pass is almost flat from 1 kb to 2 kb (3.9 ms) and then grows roughly linearly, doubling from 5 kb (7.2 ms) to 10 kb (15.8 ms) (Table VI).

### B. Exhaustive ISM

A scan of $n$ positions with a $\Delta$ window of $W$ bases and $F$ bases of flank evaluates $1 + 3n$ sequences of length $L = n + 2(W + F)$, so

$$T_{\text{ISM}}(n) = (1 + 3n)\, T_f(n + 2W + 2F) = O\big(n\, L \log L\big), \tag{9}$$

which grows as $O(n^2 \log n)$ once the region dominates the flank. Unlike CNNs, a long-convolution model offers no locality to exploit: because of causality a variant at $v$ changes every forward state at positions $\ge v$ and every reverse state at positions $\le v$, so the fastISM strategy of recomputing a small cone [15] does not apply.

### C. Speculative ISM

The draft costs $O(Nr)$ for a neighbourhood of $r = 21$ bases, i.e. $O(N)$. Selecting the top $K$ costs $O(N \log N)$ with the sort we use, or $O(N)$ with linear-time selection [41], [42]. Both are negligible next to a single forward pass for the $N = 192$ candidates we scan. Verification needs $\lceil K/B \rceil$ batched passes. If $t_1$ is the time per sequence of an unbatched pass and $t_B$ the time per sequence in a batch of $B$, then

$$T_{\text{seq}} \approx (1 + N)\, t_1, \qquad T_{\text{bat}} \approx (1 + N)\, t_B, \tag{10}$$

$$T_{\text{spec}}(K) \approx t_1 + K t_B + T_{\text{draft}} \approx (1 + K)\, t_B. \tag{11}$$

This gives a simple cost model for the speedups:

$$S_{\text{seq}}(K) = \frac{T_{\text{seq}}}{T_{\text{spec}}(K)} \approx \frac{1 + N}{1 + K} \cdot g, \qquad S_{\text{bat}}(K) \approx \frac{1 + N}{1 + K}, \tag{12}$$

with batching gain $g = t_1/t_B = T_{\text{seq}}/T_{\text{bat}}$. The model makes a point that is easy to miss: against a batched exhaustive baseline, speculation saves exactly the passes it skips and nothing more. Any speedup beyond $(1 + N)/(1 + K)$ comes from batching, which exhaustive ISM can use too. Section XII shows that Eq. (12) predicts our measurements within 1.4%.

### D. Recall bounds

To judge a draft we compare it with two reference points.

Proposition 1 (Random draft). If $S$ is a uniformly random subset of $C$ with $|S| = K$, then $\mathbb{E}[\text{recall}] = K/N$ for any $H \neq \emptyset$.

Proof. Each $c \in H$ is in $S$ with probability $\binom{N-1}{K-1}/\binom{N}{K} = K/N$. By linearity of expectation $\mathbb{E}|S \cap H| = |H|K/N$; dividing by $|H|$ gives the claim. □

Proposition 2 (Oracle draft). For any draft, recall $\leq \min(1, K/|H|)$, with equality for a draft that ranks all of $H$ first.

Proof. $|S \cap H| \leq \min(|S|, |H|) = \min(K, |H|)$, and ranking $H$ first attains the bound. □

A useful draft should therefore lie well above $K/N$ and as close as possible to $\min(1, K/|H|)$. Because $|H|$ varies between windows, we average the oracle bound per window.

## X. Interactive System

### A. Scoring service

A FastAPI [43] application exposes a WebSocket endpoint, /ws/splice-session, with the messages in Table V. Each connection holds a session with its own edited copy of the region. Edits are validated (position in range, reference base correct, alternative in {A,C,G,T}) and applied to the session immediately, because they are cheap and must never be lost. Scoring is expensive and runs on a single GPU worker thread. Every request receives a generation number, and a request that is still queued when a newer one arrives is skipped before it reaches the GPU, so stale work is avoided rather than merely hidden; a request already running finishes but is reported as cancelled. Reference predictions are cached per window, so an edit costs one mutant prediction, i.e. a forward and a reverse-complement pass over the window plus 1 kb of flank on each side. Each response carries per-base reference and mutant probabilities, the four Δ tracks and the measured latency. The scan message runs speculative ISM and returns unverified candidates as null rather than zero, together with the acceptance rate, the passes saved and the speedup, whose basis is stated in the response.

TABLE V
WebSocket protocol of the scoring service

| Message | Effect |
|---|---|
| hello | sent on connect: engine and region (sequence, exons, coordinates) |
| mutate | apply one SNV to the session, then rescore a window |
| score | rescore a window $[s, e)$ without editing |
| reset | restore the reference sequence |
| scan | speculative ISM over $[s, e)$ with budget $K$ and threshold $\tau$ |
| ping | liveness check, answered with pong |

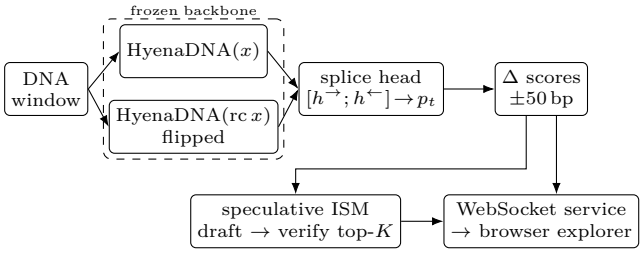



Fig. 2. Svelto-DNA overview. The frozen backbone runs on both strands; a trained head predicts per-base site probabilities, from which Δ scores, speculative scans and the interactive service are built.

### B. Browser explorer

The explorer is a Next.js application. It shows a virtualised 10 kb sequence ribbon, reference and mutant probability tracks, the four Δ tracks and live telemetry from the service. A three.js scene provides a continuous zoom from a whole-brain view, motivated by the MAPT example, down to the base pair. When the user edits a base, the server's response updates the tracks in place. The explorer sustains 57–60 frames per second on the evaluation laptop. Accessibility is checked automatically with axe, and end-to-end tests run in Playwright. By default the demo shows a synthetic 10 kb region whose sequence is generated identically on client and server; the service can equally serve real GRCh38 windows.

## XI. Experimental Setup

Hardware and software. All experiments ran on one laptop with an NVIDIA GeForce RTX 5060 Laptop GPU (8 GB), an AMD Ryzen AI 7 350 CPU and 22 GB of usable system memory, using Python 3.11, PyTorch 2.14 with CUDA 13.0 [44], Hugging Face Transformers 5.19 [45] and FastAPI 0.143.

Measurement conditions. Laptop GPUs change clock speeds with the power source and the platform power profile, and the overhead-bound batch-1 path is affected most. We therefore record the power state with each timing experiment and state it in every caption. The ablation (Fig. 3) and the batch-scaling measurement (Table XI) ran on battery in the low-power profile. The power state of the 40-window benchmark (Table IX) was not recorded. Recall, acceptance rate and all prediction metrics do not depend on hardware.

Timing protocol. All GPU timings synchronise the device before and after the timed region, follow a warm-up that excludes CUDA initialisation, and report medians unless stated otherwise. The backbone profile uses 20 iterations per length, the Δ calculator 200 on the GPU and 50 on the CPU, and batch scaling 20 per batch size.

Speculative ISM protocol. Each scan covers a region of 64 positions (192 candidates) with $W = 50$ and $F$ bases of real flank on each side, so the evaluated sequence has

TABLE VI
One fp16 HyenaDNA-small pass on the RTX 5060 Laptop GPU (20 iterations per length)

| Length (bp) | Median (ms) | p95 (ms) | Memory (MB) | Throughput (M bp/s) |
|---|---|---|---|---|
| 1,024 | 3.91 | 6.54 | 67 | 0.24 |
| 2,048 | 3.94 | 4.58 | 89 | 0.50 |
| 5,000 | 7.19 | 7.29 | 154 | 0.70 |
| 10,000 | 15.79 | 16.10 | 262 | 0.63 |

$64 + 2(50 + F)$ bases. The benchmark uses 40 held-out windows with $F = 1000$ (2,164 bp), half centred on an annotated site and half at a random position inside the labelled block (seed 7). The ablation uses 16 windows for each $F \in \{250, 500, 1000\}$ (664, 1,164 and 2,164 bp; seed 11). Verification uses batches of $B = 4$, the size at which the GPU saturates (Table XI). A variant is high-impact if $\Delta \geq 0.2$.

Reproducibility. Every number in this paper is produced by a committed script from public data and written to a JSON file in the repository: the evaluation harness, the speculative benchmark, the ablation, the case study, and a script that derives the cost-model and recall-bound statistics. Tables in the repository are regenerated from those files.

## XII. Results

### A. Backbone efficiency

Table VI profiles one fp16 backbone pass. A 10 kb window, the context SpliceAI uses, takes 15.8 ms and 262 MB of GPU memory, and throughput peaks at about 0.7 M bases per second around 5 kb. Below 2 kb the latency is set by fixed overheads, which is why short scans gain little from shorter sequences and, as shown below, little from batching.

### B. Splice-site prediction

Table VII reports held-out test performance over 62.7 M positions. ROC-AUC is near 1 for both classes, but under a 559:1 imbalance it is dominated by easy negatives and says little about usefulness [46]. PR-AUC (0.630 donor, 0.552 acceptor) and top-$k$ accuracy are more informative. Top-$k$ accuracy, as defined by SpliceAI, is the fraction of true sites among the $k$ highest-scoring positions, where $k$ is the number of true sites; we also report it at $2k$. The untrained head scores PR-AUC 0.0007, which is chance level. Table VIII shows precision and recall at the usual cut-offs. At 0.5 the model recovers 90% of donors and 86% of acceptors, but with low precision, so it is better suited to ranking positions than to calling sites with a fixed threshold.

For context, SpliceAI-10k reports top-$k$ accuracy around 0.95 on its own test set [8]. That model is trained end to end with 10 kb of context, whereas ours is a frozen 3.3 M-parameter backbone with 1 kb flanks and a 0.2 M-parameter head trained for 14 minutes. The comparison is not like for like, and we present our model as a baseline that is good enough to drive the interactive and speculative experiments, not as a competitor.

TABLE VII
Splice-site prediction on held-out chromosomes 1, 3, 5, 7, 9 (62.7 M positions, 56,008 donors, 56,011 acceptors)

| | Donor | Acceptor |
|---|---|---|
| ROC-AUC | 0.9989 | 0.9984 |
| PR-AUC | 0.6295 | 0.5520 |
| Top-$k$ accuracy | 0.604 | 0.542 |
| Top-$2k$ accuracy | 0.810 | 0.738 |
| Untrained head, PR-AUC | 0.0007 | |

TABLE VIII
Precision (P) and recall (R) of site calls at fixed probability thresholds (test set)

| | Donor | | | Acceptor | | |
|---|---|---|---|---|---|---|
| Threshold | P | R | F1 | P | R | F1 |
| 0.2 | 0.167 | 0.962 | 0.284 | 0.136 | 0.945 | 0.238 |
| 0.5 | 0.287 | 0.897 | 0.435 | 0.246 | 0.859 | 0.382 |
| 0.8 | 0.466 | 0.757 | 0.577 | 0.424 | 0.679 | 0.522 |

### C. Speculative ISM and the cost model

Table IX summarises the 40-window benchmark. Exact batched verification is only 1.2× faster than sequential ISM. At $K = 96$ the speculative scan is 2.38× faster than sequential ISM and 1.99× faster than batched exhaustive ISM, and it finds 83% of the high-impact variants on average (57% in the worst window). At $K = 24$ it is 9.1× faster but finds only 31%.

Table X compares these speedups with Eq. (12), using the measured batching gain $g = 1346/1126 = 1.195$. The prediction is within 1.4% at every budget, and the speedups over batched exhaustive ISM equal the pass ratio $(1 + N)/(1 + K)$ almost exactly. The interpretation is simple: speculation does not make individual passes cheaper, it skips passes, and its speedup is the factor by which it reduces their number times whatever batching gives. Since batching also benefits exhaustive ISM, the fair headline comparison is against batched exhaustive ISM.

### D. Why batching barely helps

Table XI measures the time per sequence of a full prediction (forward and reverse-complement passes plus head) on 2,164 bp inputs as the batch grows. The per-sequence cost falls by only 25% from batch 1 to batch 4 and then rises again, settling at about 10 ms. HyenaDNA-small has just 3.3 M parameters and $D = 256$, so a single 2 kb sequence already keeps the GPU busy with several milliseconds of FFT and element-wise work, and larger batches add memory traffic without adding useful parallelism. This is the opposite of the regime in which speculative decoding was developed, where a large model

TABLE IX
Speculative ISM on 40 held-out windows (64 bp region, 192 candidates, 2,164 bp of sequence, $\Delta \geq 0.2$). Speedup vs. sequential ISM; median times; power state not recorded.

| Method | ms | Speedup | Recall | Min. rec. | $\alpha$ |
|---|---|---|---|---|---|
| Sequential | 1346 | 1.00× | 1.00 | 1.00 | – |
| Batched exhaustive | 1126 | 1.20× | 1.00 | 1.00 | – |
| Speculative $K$=96 | 567 | 2.38× | 0.83 | 0.57 | 0.26 |
| Speculative $K$=48 | 287 | 4.68× | 0.51 | 0.00 | 0.30 |
| Speculative $K$=24 | 148 | 9.10× | 0.31 | 0.00 | 0.38 |

TABLE X
Cost model (Eq. 12) vs. measurement, $N = 192$, $g = 1.195$

| $K$ | $\frac{1+N}{1+K}$ | Pred. $S_{\text{seq}}$ | Meas. $S_{\text{seq}}$ | Error | Meas. $S_{\text{bat}}$ | Saved |
|---|---|---|---|---|---|---|
| 96 | 1.99 | 2.38 | 2.38 | −0.1% | 1.99 | 49.7% |
| 48 | 3.94 | 4.71 | 4.68 | −0.5% | 3.92 | 74.6% |
| 24 | 7.72 | 9.23 | 9.10 | −1.4% | 7.61 | 87.0% |

is memory-bound at batch 1 and verifying many tokens costs about the same as generating one.

## E. Sequence length and budget

Fig. 3 varies the sequence length and the budget. Recall depends on $K$ but hardly on length: averaged over all 48 window–length runs it is 0.28, 0.39, 0.60 and 0.87 at $K = 12, 24, 48, 96$. The ablation's timings were recorded on battery in the low-power profile, which slows the overhead-bound sequential path disproportionately. Its batching gain is therefore larger (1.4–3.1×) than in Table IX, and so are its absolute speedups, consistent with Eq. (12); the figure should be read for its trends rather than its absolute values. The batching gain shrinks as sequences get longer, as expected when per-sequence work grows relative to fixed overheads.

## F. Draft quality against random and oracle drafts

Fig. 4 places the heuristic draft between the two bounds of Section IX, using the ablation's 48 runs. Windows contain on average 32.6 high-impact candidates out of 192 (39.3 in windows centred on a site, 25.9 at random positions). The draft is 4.5× better than random at $K = 12$ and 1.7× better at $K = 96$, so it carries real information. It reaches only 51–64% of the oracle at $K \leq 48$, however, and 87% at $K = 96$, where the oracle already has perfect recall. At small budgets the draft spends its slots on candidates that hit predicted sites but are not high-impact, and the acceptance rate of 0.26–0.38 in Table IX shows the same thing. Recall is similar in windows centred on a site and in random windows (0.84 and 0.91 at $K = 96$), so the draft is not tuned to the site-centred case.

## G. Interactive latency

Through the WebSocket service, a 1 kb edit (a mutant prediction over the window plus flanks, followed by $\Delta$ computation and serialisation) takes a median round trip of 12.5 ms (p95 13.9 ms), measured by the service's latency test with the real backbone and head. This is well below the commonly cited 100 ms limit for a response to feel immediate, so a user can edit bases and watch the splice landscape respond as they type.

TABLE XI
Batch scaling of one full prediction (fwd + rc passes + head) on 2,164 bp inputs; median of 20; battery, low-power profile

| Batch | Total (ms) | Per sequence (ms) | Relative |
|---|---|---|---|
| 1 | 11.03 | 11.03 | 1.00 |
| 2 | 17.29 | 8.64 | 0.78 |
| 4 | 33.06 | 8.26 | 0.75 |
| 8 | 78.52 | 9.82 | 0.89 |
| 16 | 159.93 | 10.00 | 0.91 |
| 32 | 318.84 | 9.96 | 0.90 |
| 64 | 640.58 | 10.01 | 0.91 |

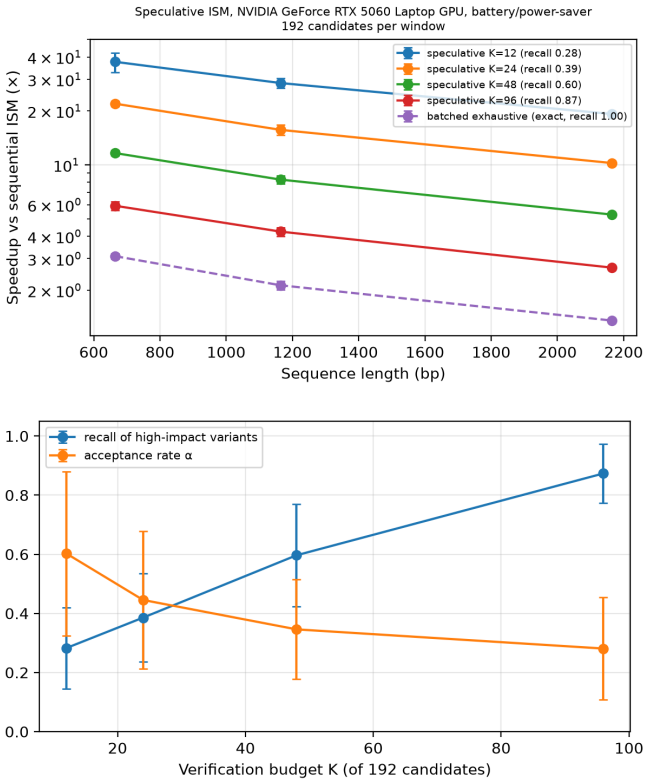



Fig. 3. Top: speedup over sequential ISM against sequence length (mean ± s.d. over 16 windows); the legend gives each budget's recall. Bottom: recall and acceptance rate against the budget $K$. Timings recorded on battery in the low-power profile.

## H. ClinVar case studies

We scored the eight ClinVar pathogenic variants of Table IV with our own model. Minus-strand variants are scored in transcript orientation, as in training, and their peak offsets are converted back to genomic orientation. Table XII and Fig. 5 show the results. Our model calls the correct site-loss component for all eight variants, with

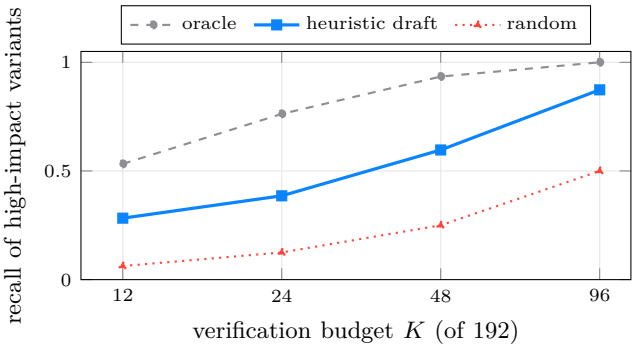



Fig. 4. Recall of the heuristic draft compared with a uniform-random draft (Proposition 1) and an oracle draft (Proposition 2), averaged over the 48 ablation runs.

peak $\Delta$ between 0.840 and 0.995 (mean 0.944, against 0.973 for SpliceAI; mean absolute difference 0.051). Three variants lie on training chromosomes, so their sites were seen as labels during training; three are on validation chromosomes, used only for checkpoint selection; and two are on held-out test chromosomes. The two test variants, ABHD5 ($\Delta_{\mathrm{AL}} = 0.962$) and ABCA1 ($\Delta_{\mathrm{DL}} = 0.903$), behave like the rest.

Peak positions differ from SpliceAI's by 1–2 bp. This is not an error. We label the first base of the dinucleotide, while SpliceAI labels the exon-side boundary base, which in transcript orientation lies one base before our donor label and two bases after our acceptor label. All eight offsets differ by exactly this convention shift, which also confirms that our strand handling is correct.

Secondary components show where the models disagree. For ABCC2 our model predicts a small acceptor loss ($\Delta_{\mathrm{AL}} = 0.32$) that SpliceAI does not (0.001), while both predict a donor gain of similar size (0.22 versus 0.245). For AAAS SpliceAI predicts an acceptor loss of 0.28 that we miss. Such disagreements are visible to a user of the explorer, who can inspect all four tracks.

Fig. 6 shows two variants in detail. In ABCA1 (test chromosome, minus strand) the reference donor peak of 0.90 vanishes in the mutant. Gradient×input saliency [31] of the reference donor concentrates on the dinucleotide and the three exonic positions before it, matching the MAG|GT consensus. ABAT shows the same pattern on the plus strand.

## XIII. Discussion

### A. Speed versus recall

Speculative verification is exact for the candidates it verifies, but in variant scanning the cost of a missed high-impact variant is clinical, not cosmetic. On our hardware the honest summary is: 1.2× with no missed variants, or about 2.4× while missing about one high-impact variant in six on average and up to 43% in the worst window. Unlike token-level speculative decoding, where rejected drafts are regenerated and outputs stay identical, an unverified variant here is simply unknown. We therefore recommend that speculative methods for genomic models always report recall against exhaustive scanning, and that interfaces present unverified candidates as such.

TABLE XII
Our model on eight ClinVar pathogenic splice variants. Split: the chromosome's role in training. Offsets (ours/SpliceAI) are genomic; all pairs differ by the labelling convention.

| Gene | Chr. | Split | Str. | Comp. | Ours | SpliceAI | Offsets |
|---|---|---|---|---|---|---|---|
| ABAT | 16 | train | + | DL | 0.937 | 0.996 | 0/−1 |
| ABCC2 | 10 | val | + | DL | 0.957 | 0.995 | −1/−2 |
| ABCG8 | 2 | val | + | AL | 0.840 | 0.936 | −1/+1 |
| ABHD5 | 3 | test | + | AL | 0.962 | 0.981 | 0/+2 |
| AAAS | 12 | train | − | DL | 0.969 | 0.985 | 0/+1 |
| AAGAB | 15 | train | − | DL | 0.986 | 0.986 | 0/+1 |
| AARS2 | 6 | val | − | DL | 0.995 | 0.905 | 0/+1 |
| ABCA1 | 9 | test | − | DL | 0.903 | 0.997 | 0/+1 |
| Mean | | | | | 0.944 | 0.973 | |

### B. When would speculation pay off?

The cost model (Eq. 12) shows what would have to change. On hardware or models where batching is efficient ($g \gg 1$), exhaustive batched ISM becomes cheap and the case for speculation weakens further. Speculation pays off when passes are expensive and cannot be amortised, as with large backbones, long contexts or CPU-only deployment, and when the draft has high recall at small $K$. Our draft leaves a large gap to the oracle at $K \leq 48$ (Fig. 4). A learned drafter trained on exhaustive ISM labels, which are cheap to generate offline, is the most promising way to close it. Another route would make speculation lossless: if a cheap certificate could bound $\Delta(c) < \tau$ for the skipped candidates, for example a Lipschitz bound of the head around the reference features, a scan could skip them with a guarantee instead of a heuristic.

### C. Reverse-complement fusion as a cheap fix for causality

Running a causal model on both strands doubles the cost of a pass but needs no retraining of the backbone, and our measurements confirm that the second pass supplies the downstream context the first lacks. Bidirectional, reverse-complement-equivariant models such as Caduceus [25] build this into the architecture and may give better features at the same cost. For frozen causal backbones, two-strand fusion is a reasonable default for any task in which downstream sequence matters.

### D. Frozen backbone, small head

A frozen 3.3 M-parameter backbone with a 0.2 M-parameter head reaches a mean test PR-AUC of 0.59 after 14 minutes of training, but it remains well behind SpliceAI. Three factors probably account for most of the gap: the 1 kb flank versus SpliceAI's 5 kb on each side, the absence of fine-tuning, and the small backbone. The training curve (Fig. 1) is still rising slowly, so longer

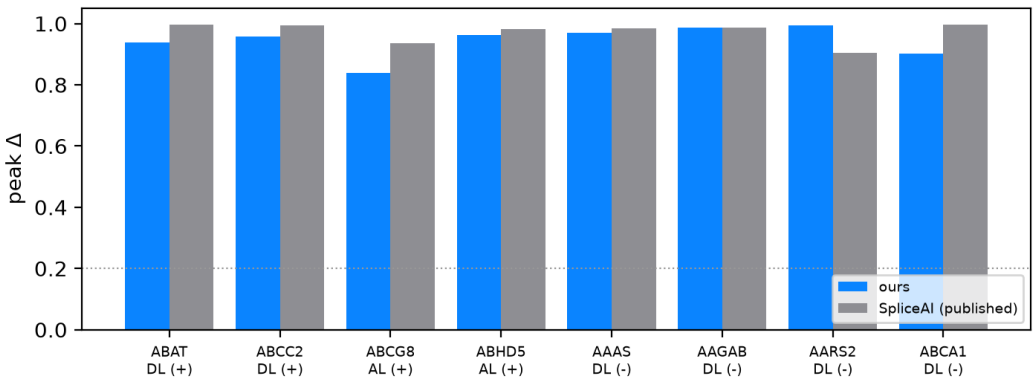



Fig. 5. Peak Δ of the correct component for eight ClinVar pathogenic splice variants: our model (blue) and SpliceAI as published (grey). The dotted line is the 0.2 high-recall cut-off.

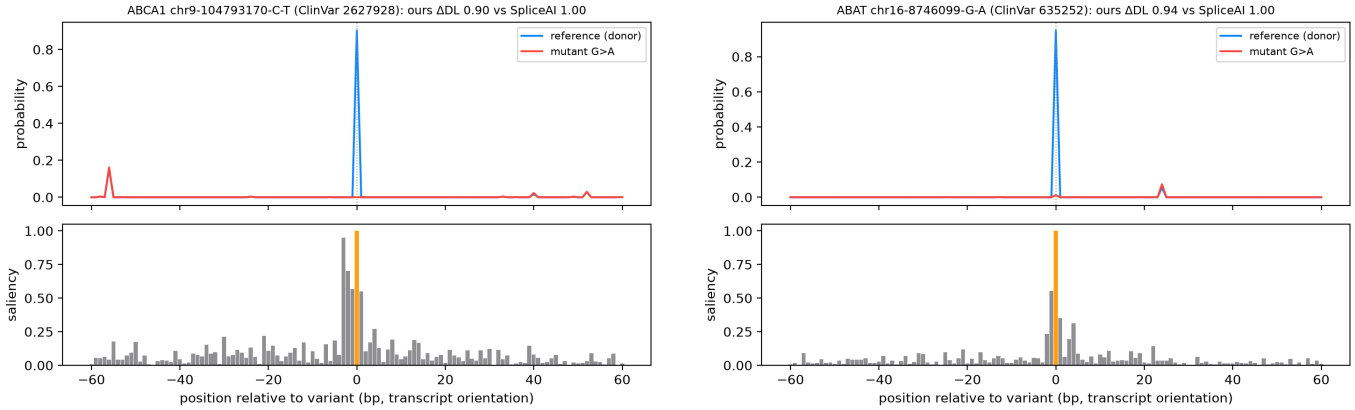



Fig. 6. Left: ABCA1 chr9:104,793,170 C>T (ClinVar 2627928; held-out test chromosome; minus strand, shown in transcript orientation, where the variant reads G>A). Right: ABAT chr16:8,746,099 G>A (ClinVar 635252; training chromosome). Top panels: reference and mutant donor probability. Bottom panels: gradient×input saliency of the reference donor, with the variant highlighted.

training would add a little, but context and capacity are likely to matter more. For an interactive tool the trade is attractive: the whole model fits easily in laptop GPU memory and an edit is rescored in 12.5 ms.

### E. Silent data errors

Both ClinVar errors in Section V would have passed unnoticed in aggregate statistics: the first admitted every SNV in ClinVar regardless of distance, and the second moved 86,949 variants into the positive class. They were found only by checking individual records against their source text. We encourage anyone building on ClinVar to test their parser against the current wording of the clinical-significance field.

## XIV. Limitations and Threats to Validity

Data. Paralogs are not removed from the test set, so some test genes may resemble training genes and inflate test metrics. Labels come from one canonical transcript per gene, so sites used only by other isoforms count as negatives, which depresses measured precision.

Model. The model sees only 1 kb of flank, well below SpliceAI's 10 kb of context, and the backbone is not fine-tuned. The case study covers eight variants, three of them on training chromosomes; it illustrates behaviour but is not a clinical validation.

Timing. All timings come from one laptop GPU, and wall-clock results depend on its power state. The ablation and batch-scaling measurements ran on battery in a low-power profile, and the power state of the main benchmark was not recorded. Recall and acceptance rate are unaffected, and the cost model explains how timings scale with the batching gain, but absolute speedups should be re-measured on mains power before they are compared with other work.

Scope of the speculative study. We evaluated one heuristic and one gradient-based draft, one model size and regions of 64 positions. Larger regions increase $N$ and could favour speculation if the draft's recall holds, which we have not tested.

Demo. The browser demo shows a synthetic region by default, although the service can serve real GRCh38

windows.

## XV. Ethical Considerations

Svelto-DNA is a research and teaching tool. Its predictions are not validated for clinical use, and its splice-site model is measurably weaker than established predictors. Speculative scans deliberately leave candidates unscored, and a user who mistook "not verified" for "benign" could miss a pathogenic variant; for this reason the service never reports unverified candidates as zero. All data used are public (GENCODE, Ensembl, ClinVar) and contain no individual-level information, and the service processes only sequences supplied by the user.

## XVI. Conclusion

Svelto-DNA turns a frozen genomic foundation model into an interactive splice-disruption tool with validated $\Delta$ scores and edits rescored in 12.5 ms. Two-strand fusion makes a causal backbone usable for splice prediction, and the model calls the correct site loss for all eight ClinVar pathogenic variants we examined. For speculative in-silico mutagenesis, a simple cost model explains the measured speedups almost exactly: against batched exhaustive ISM, speculation saves only the passes it skips, and on small models and consumer GPUs batching adds little. The result is a tunable trade between speed and recall rather than a free speedup. Our heuristic draft is far better than random but leaves a large gap to an oracle at small budgets. Next steps are a learned drafter trained on exhaustive ISM labels, certified skipping, longer context, bidirectional backbones, and measurements on server-class GPUs where batching may amortise better.

## Code and Data Availability

Code, scripts and measured results are in the Svelto-DNA repository. Every number in this paper is produced by a committed script from public GENCODE, Ensembl and ClinVar data and stored as JSON alongside the scripts.

## References


[1] Y. Lee and D. C. Rio, "Mechanisms and regulation of alternative pre-mRNA splicing," Annual Review of Biochemistry, vol. 84, pp. 291–323, 2015.

[2] Z. Wang and C. B. Burge, "Splicing regulation: From a parts list of regulatory elements to an integrated splicing code," RNA, vol. 14, no. 5, pp. 802–813, 2008.

[3] M. M. Scotti and M. S. Swanson, "RNA mis-splicing in disease," Nature Reviews Genetics, vol. 17, pp. 19–32, 2016.

[4] H. Y. Xiong, B. Alipanahi, L. J. Lee et al., "The human splicing code reveals new insights into the genetic determinants of disease," Science, vol. 347, no. 6218, p. 1254806, 2015.

[5] M. Hutton, C. L. Lendon, P. Rizzu et al., "Association of missense and 5'-splice-site mutations in tau with the inherited dementia FTDP-17," Nature, vol. 393, pp. 702–705, 1998.

[6] S. Richards, N. Aziz, S. Bale et al., "Standards and guidelines for the interpretation of sequence variants: A joint consensus recommendation of the American College of Medical Genetics and Genomics and the Association for Molecular Pathology," Genetics in Medicine, vol. 17, no. 5, pp. 405–424, 2015.

[7] L. C. Walker et al., "Using the ACMG/AMP framework to capture evidence related to predicted and observed impact on splicing: Recommendations from the ClinGen SVI splicing subgroup," American Journal of Human Genetics, vol. 110, no. 7, pp. 1046–1067, 2023.

[8] K. Jaganathan, S. Kyriazopoulou Panagiotopoulou, J. F. McRae et al., "Predicting splicing from primary sequence with deep learning," Cell, vol. 176, no. 3, pp. 535–548, 2019.

[9] T. Zeng and Y. I. Li, "Predicting RNA splicing from DNA sequence using Pangolin," Genome Biology, vol. 23, p. 103, 2022.

[10] Y. Ji, Z. Zhou, H. Liu, and R. V. Davuluri, "DNABERT: Pre-trained bidirectional encoder representations from Transformers model for DNA-language in genome," Bioinformatics, vol. 37, no. 15, pp. 2112–2120, 2021.

[11] Z. Zhou, Y. Ji, W. Li, P. Dutta, R. Davuluri, and H. Liu, "DNABERT-2: Efficient foundation model and benchmark for multi-species genome," in Proc. International Conference on Learning Representations (ICLR), 2024.

[12] H. Dalla-Torre, L. Gonzalez, J. Mendoza-Revilla et al., "Nucleotide Transformer: Building and evaluating robust foundation models for human genomics," Nature Methods, vol. 22, pp. 287–297, 2025.

[13] E. Nguyen, M. Poli, M. Faizi et al., "HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution," in Advances in Neural Information Processing Systems (NeurIPS), 2023.

[14] B. Alipanahi, A. Delong, M. T. Weirauch, and B. J. Frey, "Predicting the sequence specificities of DNA- and RNA-binding proteins by deep learning," Nature Biotechnology, vol. 33, pp. 831–838, 2015.

[15] S. Nair, A. Shrikumar, J. Schreiber, and A. Kundaje, "fastISM: Performant in silico saturation mutagenesis for convolutional neural networks," Bioinformatics, vol. 38, no. 9, pp. 2397–2403, 2022.

[16] Y. Leviathan, M. Kalman, and Y. Matias, "Fast inference from transformers via speculative decoding," in Proc. International Conference on Machine Learning (ICML), 2023.

[17] C. Chen, S. Borgeaud, G. Irving et al., "Accelerating large language model decoding with speculative sampling," arXiv preprint arXiv:2302.01318, 2023.

[18] M. Stern, N. Shazeer, and J. Uszkoreit, "Blockwise parallel decoding for deep autoregressive models," in Advances in Neural Information Processing Systems (NeurIPS), 2018.

[19] T. Cai, Y. Li, Z. Geng et al., "Medusa: Simple LLM inference acceleration framework with multiple decoding heads," in Proc. International Conference on Machine Learning (ICML), 2024.

[20] G. Yeo and C. B. Burge, "Maximum entropy modeling of short sequence motifs with applications to RNA splicing signals," Journal of Computational Biology, vol. 11, no. 2–3, pp. 377–394, 2004.

[21] A. Vaswani, N. Shazeer, N. Parmar et al., "Attention is all you need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.

[22] M. Poli, S. Massaroli, E. Nguyen et al., "Hyena hierarchy: Towards larger convolutional language models," in Proc. International Conference on Machine Learning (ICML), 2023.

[23] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," Mathematics of Computation, vol. 19, no. 90, pp. 297–301, 1965.

[24] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv preprint arXiv:2312.00752, 2023.

[25] Y. Schiff, C.-H. Kao, A. Gokaslan, T. Dao, A. Gu, and V. Kuleshov, "Caduceus: Bi-directional equivariant long-range DNA sequence modeling," in Proc. International Conference on Machine Learning (ICML), 2024.

[26] J. Cheng, T. Y. D. Nguyen, K. J. Cygan et al., "MMSplice: Modular modeling improves the predictions of genetic variant effects on splicing," Genome Biology, vol. 20, p. 48, 2019.

[27] Broad Institute, "SpliceAI-lookup," https://spliceailookup.broadinstitute.org, accessed October 2026.

[28] M. Kircher, D. M. Witten, P. Jain et al., "A general framework for estimating the relative pathogenicity of human genetic variants," Nature Genetics, vol. 46, pp. 310–315, 2014.

[29] Ž. Avsec, V. Agarwal, D. Visentin et al., "Effective gene expression prediction from sequence by integrating long-range interactions," Nature Methods, vol. 18, pp. 1196–1203, 2021.
[30] K. Simonyan, A. Vedaldi, and A. Zisserman, "Deep inside convolutional networks: Visualising image classification models and saliency maps," arXiv preprint arXiv:1312.6034, 2013.
[31] A. Shrikumar, P. Greenside, and A. Kundaje, "Learning important features through propagating activation differences," in Proc. International Conference on Machine Learning (ICML), 2017.
[32] M. Sundararajan, A. Taly, and Q. Yan, "Axiomatic attribution for deep networks," in Proc. International Conference on Machine Learning (ICML), 2017.
[33] D. Y. Fu, H. Kumbong, E. Nguyen, and C. Ré, "FlashFFTConv: Efficient convolutions for long sequences with tensor cores," in Proc. International Conference on Learning Representations (ICLR), 2024.
[34] W. J. Kent, C. W. Sugnet, T. S. Furey et al., "The human genome browser at UCSC," Genome Research, vol. 12, no. 6, pp. 996–1006, 2002.
[35] A. Frankish, S. Carbonell-Sala, M. Diekhans et al., "GENCODE: Reference annotation for the human and mouse genomes in 2023," Nucleic Acids Research, vol. 51, no. D1, pp. D942–D949, 2023.
[36] P. W. Harrison, M. R. Amode, O. Austine-Orimoloye et al., "Ensembl 2024," Nucleic Acids Research, vol. 52, no. D1, pp. D891–D899, 2024.
[37] M. J. Landrum, J. M. Lee, M. Benson et al., "ClinVar: Improving access to variant interpretations and supporting evidence," Nucleic Acids Research, vol. 46, no. D1, pp. D1062–D1067, 2018.
[38] P. Micikevicius, S. Narang, J. Alben et al., "Mixed precision training," in Proc. International Conference on Learning Representations (ICLR), 2018.
[39] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in Proc. International Conference on Learning Representations (ICLR), 2019.
[40] L. N. Smith and N. Topin, "Super-convergence: Very fast training of neural networks using large learning rates," in Proc. SPIE Artificial Intelligence and Machine Learning for Multi-Domain Operations Applications, vol. 11006, 2019.
[41] M. Blum, R. W. Floyd, V. Pratt, R. L. Rivest, and R. E. Tarjan, "Time bounds for selection," Journal of Computer and System Sciences, vol. 7, no. 4, pp. 448–461, 1973.
[42] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.
[43] S. Ramírez, "FastAPI," https://fastapi.tiangolo.com, accessed October 2026.
[44] A. Paszke, S. Gross, F. Massa et al., "PyTorch: An imperative style, high-performance deep learning library," in Advances in Neural Information Processing Systems (NeurIPS), 2019.
[45] T. Wolf, L. Debut, V. Sanh et al., "Transformers: State-of-the-art natural language processing," in Proc. Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP), 2020, pp. 38–45.
[46] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLoS ONE, vol. 10, no. 3, p. e0118432, 2015.